IS4103
Operating Systems

Memory Management

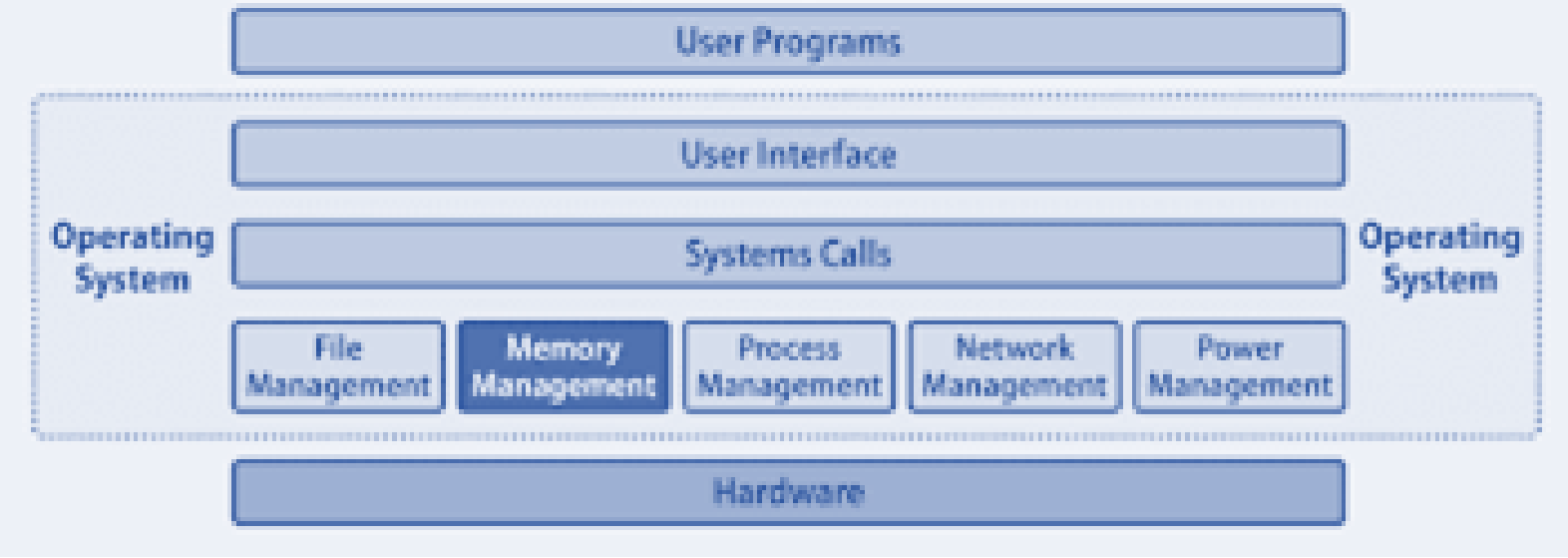
T.Kayalvizhy

Learning Objectives

Learning Outcome	**Description**
ILO3	Evaluate the performance of various scheduling algorithms

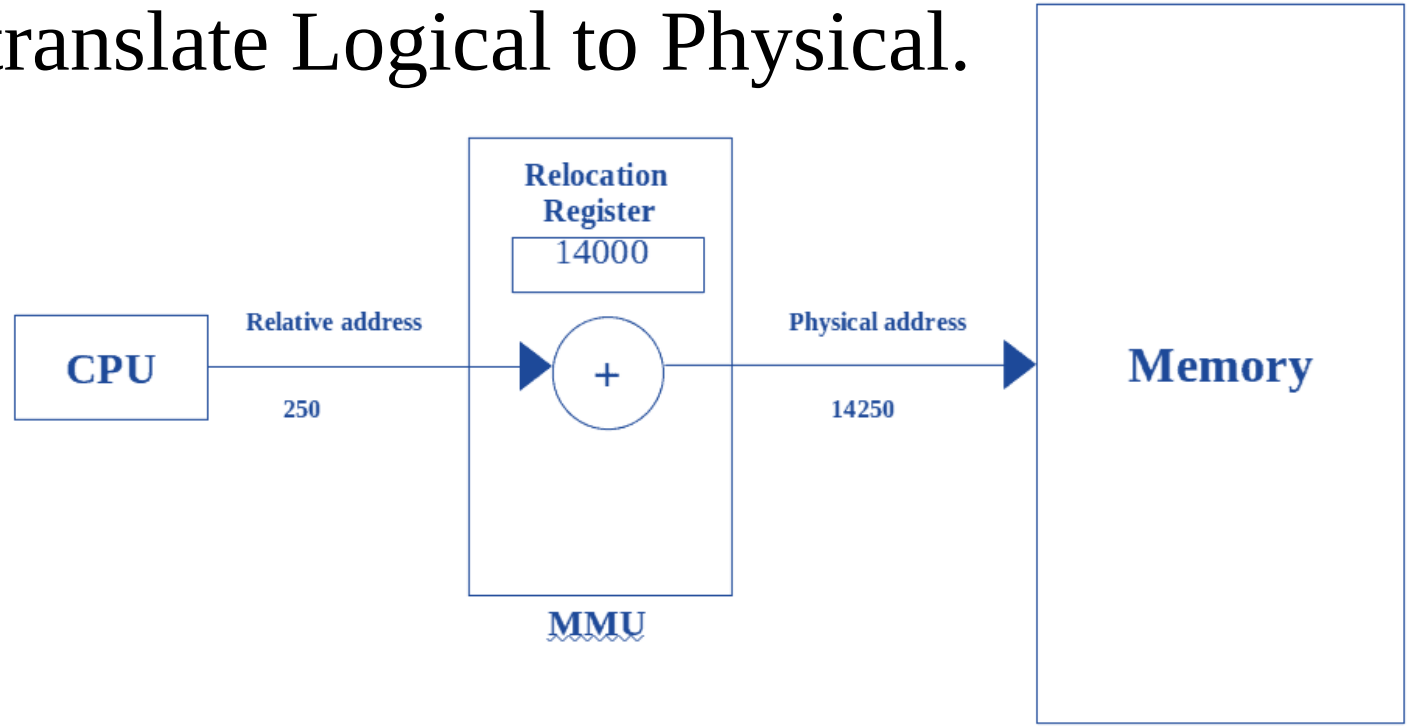
Memory Management

- Memory management is the OS function responsible for:
  - Allocating memory to programs
  - Deallocating memory after use
  - Tracking memory usage



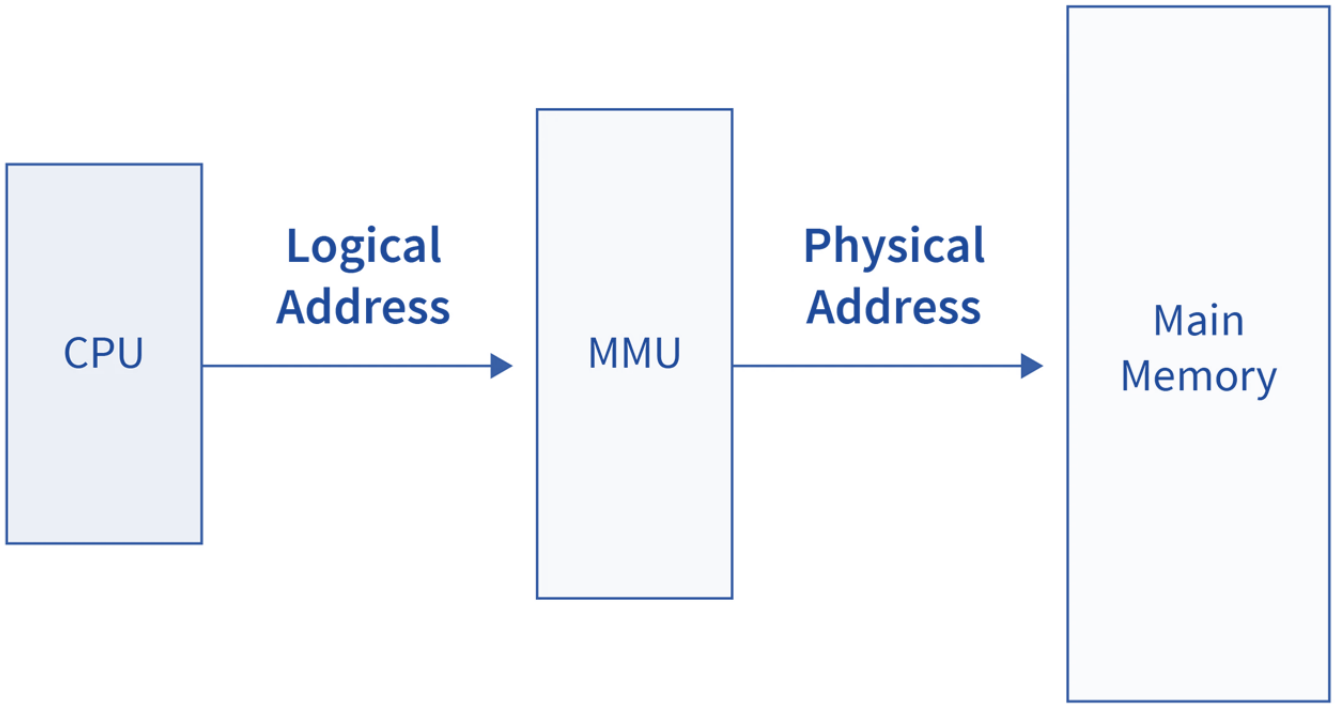
Logical vs. Physical Address Space

- Logical Address: Generated by the CPU (also called "Virtual Address"). The programmer sees this.
- Physical Address: The actual location in the memory unit (RAM).
- The Bridge: The Memory Management Unit (MMU) sits between the CPU and RAM to translate Logical to Physical.

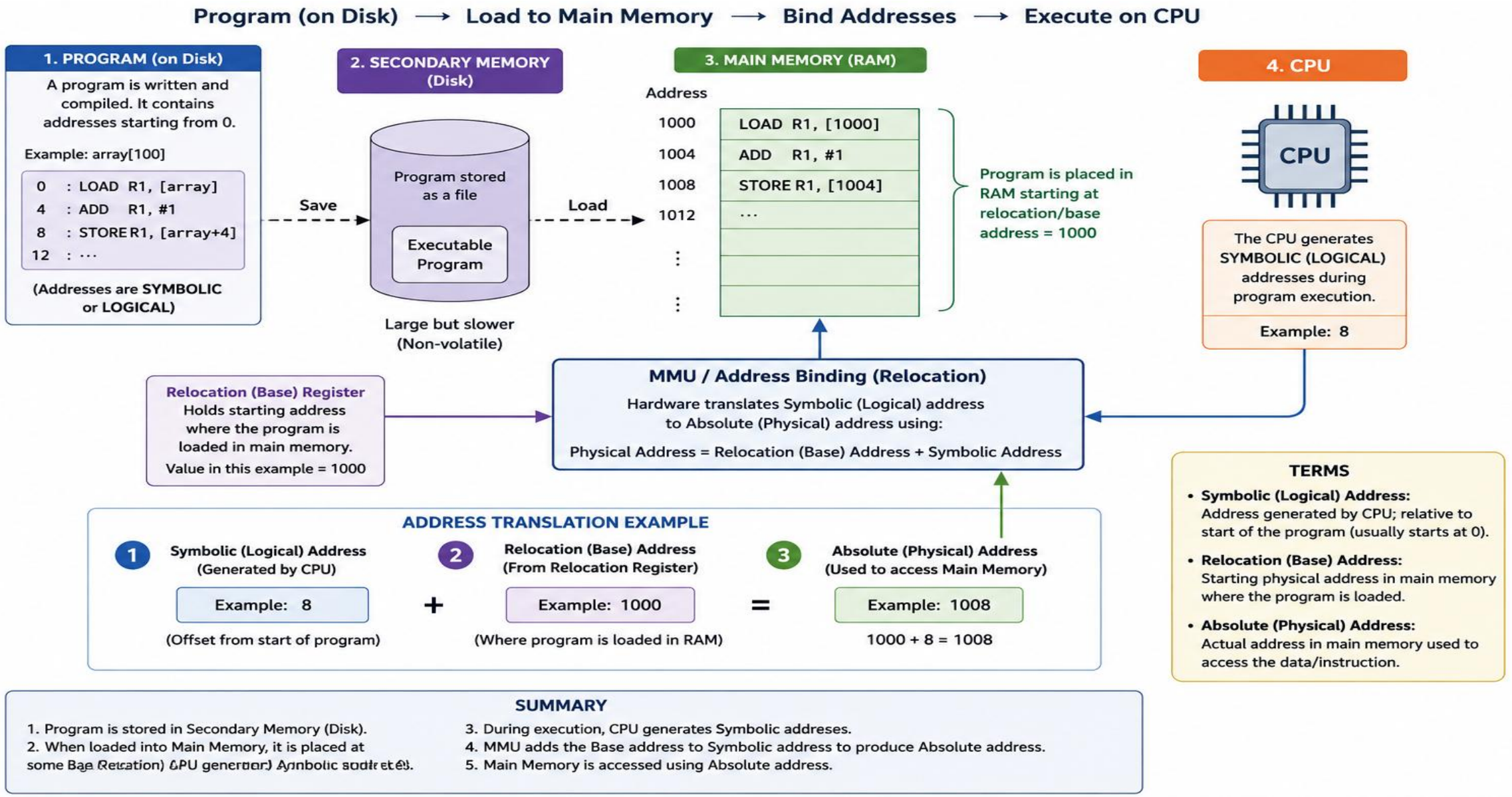


Address binding

- Address binding is the process of mapping program instructions and data to actual locations in computer memory.

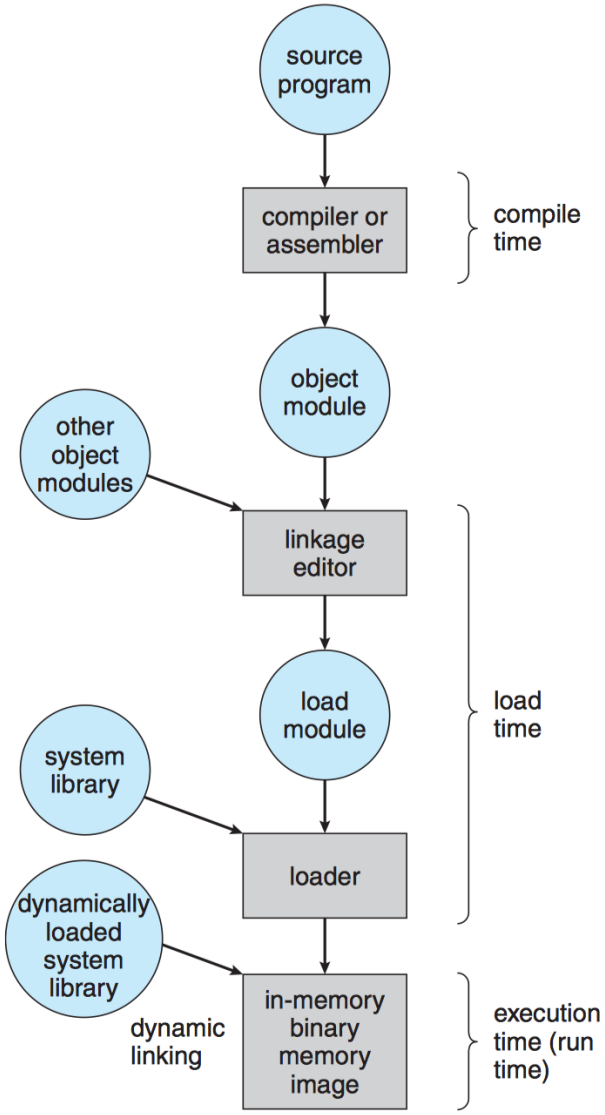


Address binding



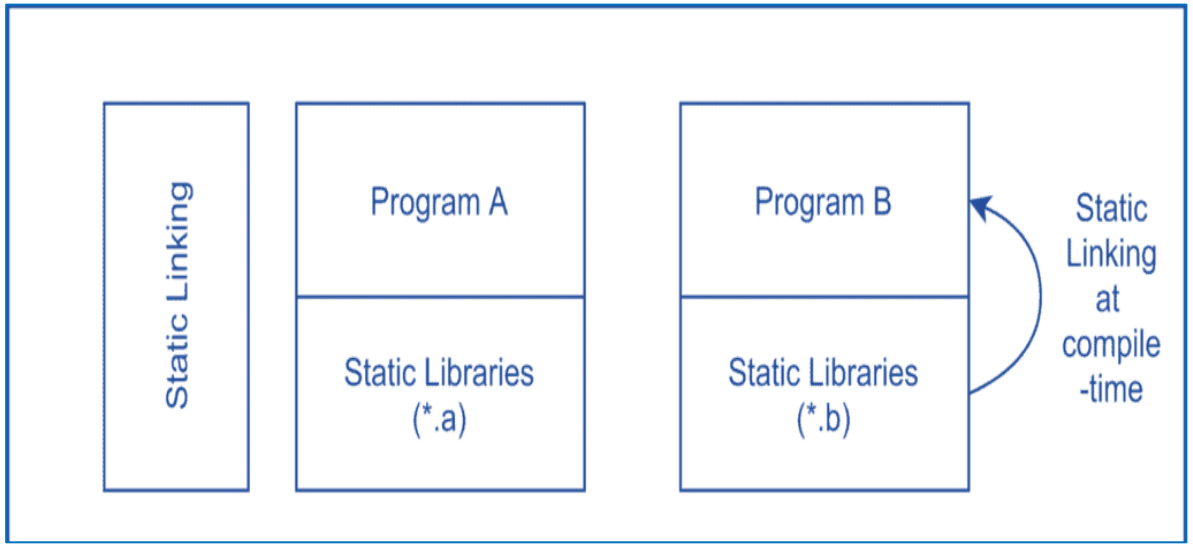
Types of Address Binding (Stages)

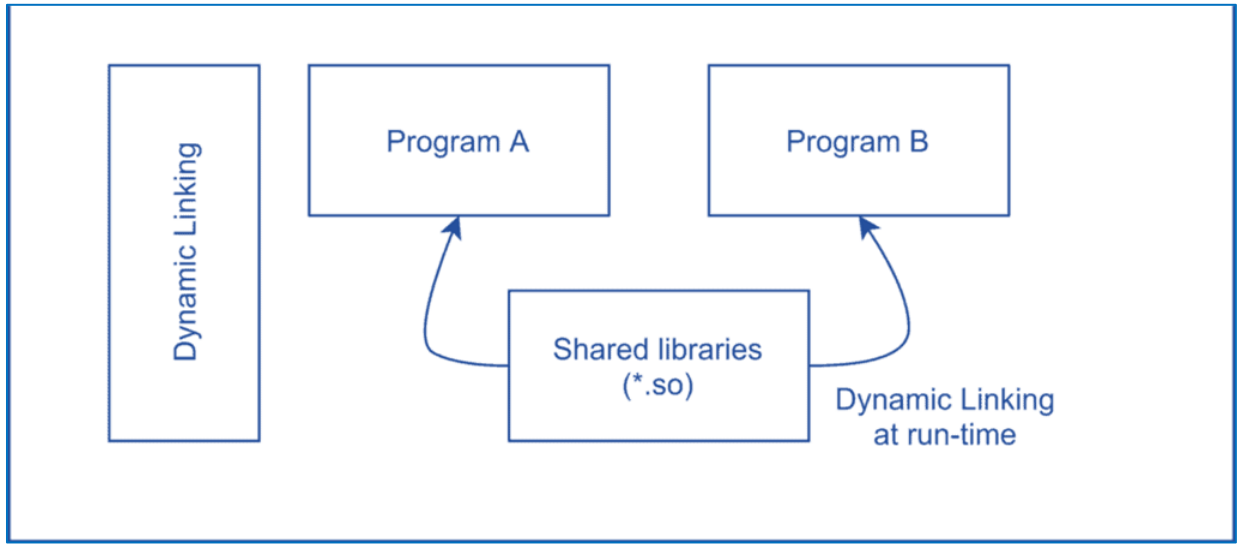
- **Compile-Time Binding**: The compiler translates symbolic addresses to absolute addresses (e.g., location 1000).
- **Load-Time Binding**: The compiler generates relocatable code. The final physical address is decided only when the program is loaded into memory.
- **Execution-Time (Run-Time) Binding**: The process can be moved during its execution from one memory segment to another. This requires hardware support (like MMU).



Dynamic Loading & Linking

- **Dynamic Loading**: A routine is not loaded until it is called. This saves space.
- **Dynamic Linking**: Rather than including library code in the executable, a "stub" is included. The actual linking happens at runtime (e.g., .DLL files in Windows).





DYNAMIC LOADING: OVERVIEW

A program is loaded into memory only when it is ready to execute.

1. PROGRAM (Source Code)

hello.c

```
#include <stdio.h>

int main() {
  printf("Hello\n");
  return 0;
}
```

(High-level language Source Code)

2. COMPILER

What happens in the Compiler?

- Checks for syntax errors
- Checks types and semantics
- Generates object code (machine code with relocatable addresses)
- Does not know final memory location

Compiler

Output: hello.o (Object Code)

```
101100  ....
010111  ....
110010  ....
```

(Relocatable machine code)

3. LINKER

What happens in the Linker?

- Combines one or more object files and library routines
- Resolves external symbols
- Produces a load module (still not loaded in memory)
- Addresses are still relative

hello.o + Libraries (e.g., printf.o)

Output: hello.exe (Load Module)

```
100101  ....
001110  ....
110011  ....
```

(Relocatable load module)

4. LOADER (Dynamic Loading)

What happens in the Loader?

- Loads the program into memory when it is ready to run
- Allocates memory for the program
- Performs address relocation (binds addresses to actual memory locations)
- Transfers control to the program's entry point

Loader

Main Memory (RAM)

1000	100101 ....
1004	001110 ....
1008	110011 ....
1012	...
...	...

(Program is loaded only now)

5. EXECUTION

CPU executes the program instruction by instruction.

CPU

Program Execution ... ...

SAMPLE PROGRAM (hello.c)

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 10, y = 20;
    printf("Sum = %d\n", add(x, y));
    return 0;
}
```

WHAT HAPPENS IN THE COMPILER (DETAIL)

1. Lexical Analysis → breaks code into tokens
2. Syntax Analysis → checks grammar (parsing)
3. Semantic Analysis → checks meaning (types, declarations, etc.)
4. Intermediate Code Generation → creates intermediate representation
5. Code Optimization → improves efficiency
6. Code Generation → produces object code (relocatable machine code)

Output: hello.o (object code with relocatable addresses and symbol table)

DYNAMIC LINKING

Libraries are linked at run-time, not at compile/link time.

Program (hello.exe) → Needs a Function (e.g., printf) → OS Loader (Loads the program into memory) → Dynamic Linker (Run-time) (Resolves external symbols) → Shared Library (printf in libc.dll / libc.so)

Dynamic Linker (Run-time) → Function Resolved and Execution Continues

How it works:

1. Program is loaded into memory.
2. When program calls an external function (e.g., printf), the call is intercepted.
3. Dynamic linker searches the required function in loaded shared libraries.
4. The address is resolved and patched (or used via a jump table).
5. Execution continues.

Benefits of Dynamic Linking

- ✓ Smaller executable size
- ✓ Libraries can be updated without recompiling the program
- ✓ Multiple programs can share the same library in memory
- ✓ Supports modularity and flexibility

EXAMPLE FLOW (Standard Library)

1. Program calls printf("Hello")
2. printf is not in the program → control goes to dynamic linker
3. Dynamic linker looks in loaded libraries (libc.so / libc.dll)
4. Finds printf address
5. Returns control to program
6. printf executes
7. Control returns to program
8. Program continues

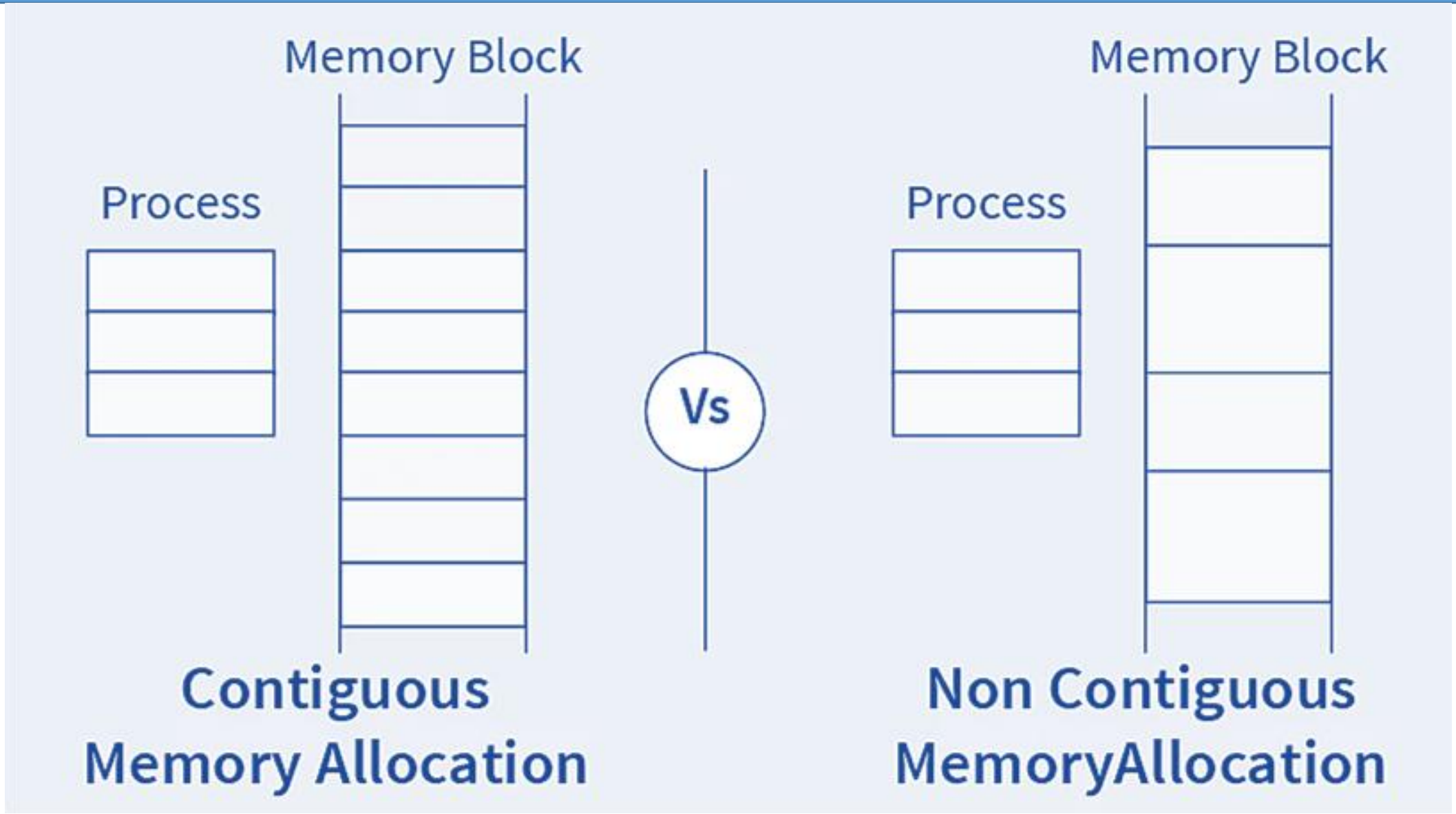
Main Memory

0x1000	Program Code
0x2000	(Unresolved) printf()
0x3000	Dynamic Linker
0x4000	libc.so / libc.dll (printf code)

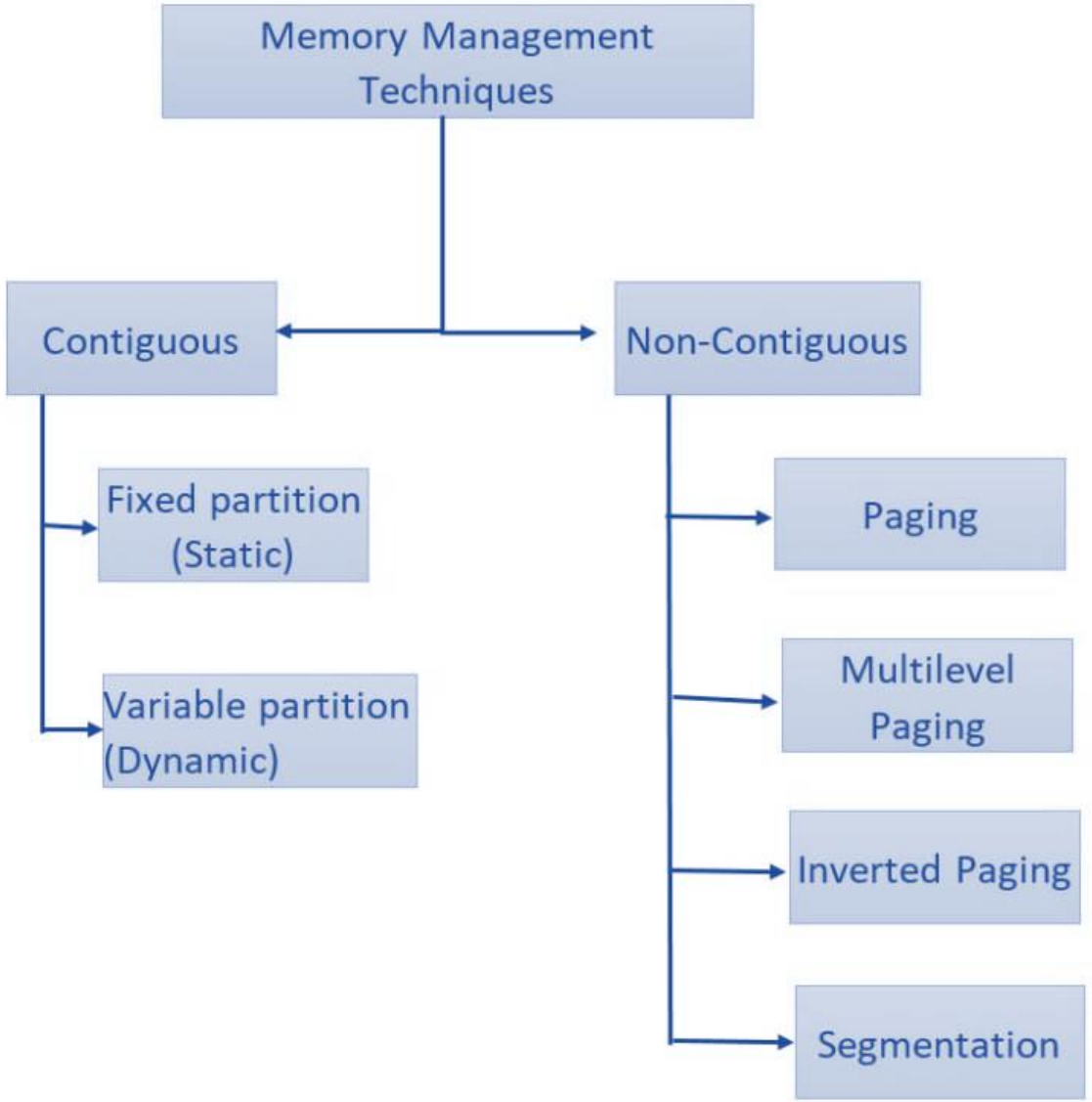
Program Calls printf() → Not in program → Dynamic Linker Resolves printf → printf() Address Found in libc → (Address returned to program)

Note: In some systems, the first call to a function goes through the dynamic linker. Later calls may go directly to the resolved address (Procedure Linkage Table - PLT).

Contiguous & Non-Contiguous Memory Allocation



Contiguous & Non-Contiguous Memory Allocation

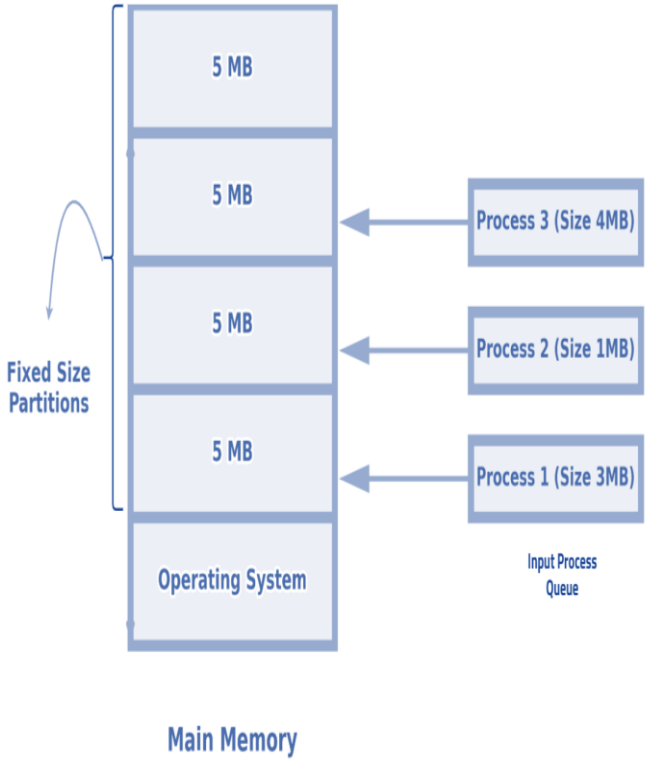


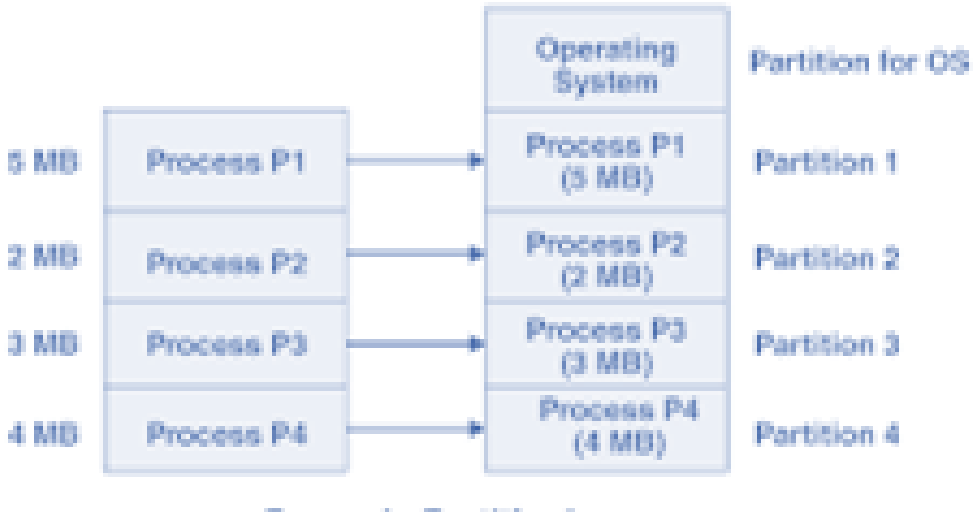
Contiguous & Non-Contiguous Memory Allocation

Feature	Contiguous Memory Allocation	Non-Contiguous Memory Allocation
Storage Style	The process is stored in one single, continuous block of RAM.	The process is divided into parts (Pages/Segments) and scattered in RAM.
Hardware	Requires simple **Base and Limit registers**.	Requires a **Page Table** or Segment Table and specialized hardware (MMU).
Fragmentation	Suffers heavily from **External Fragmentation**.	Eliminates External Fragmentation; may have tiny **Internal Fragmentation**.
OS Overhead	Low. The OS only tracks the start address and the size.	High. The OS must maintain a mapping table for every single process.
Execution	Faster address translation (simple addition).	Slightly slower due to table lookups (minimized by TLB hardware).
Swapping	Difficult, as the OS must find a single large hole to bring a process back.	Easy, as the OS can bring pieces back into any available small frames.

Contiguous Memory Allocation

- Contiguous memory allocation is an OS technique where each process is assigned a single, unbroken block of main memory.





Advantages and Disadvantages of Equal Size Partitioning

Advantages	Disadvantages
Simple implementation	Internal fragmentation
Easy allocation	Large processes cannot fit
Easy memory tracking	Poor memory utilization
Fast allocation process	External fragmentation

Equal Size Partitioning

Question

A system has:

Total memory = 1000 KB

5 equal partitions

Processes:

P1 = 120 KB
P2 = 170 KB
P3 = 220 KB
P4 = 80 KB

Find partition size, allocate processes, and calculate internal fragmentation

Advantages and Disadvantages of Variable-Size Fixed Partitioning

Advantages	Disadvantages
Better memory utilization	Small partitions may remain unused
Supports different process sizes	Still causes internal fragmentation
Reduced internal fragmentation	Complex partition selection
More flexible than equal partitioning	Difficult memory management

Fixed Partitioning

Question

Memory Partitions:

100 KB

200 KB

300 KB

500 KB

Processes:

P1 = 90 KB, P2 = 260 KB, P3 = 180 KB, P4 = 470 KB

Find partition size, unallocated processes, and calculate internal fragmentation

Overlay Technique

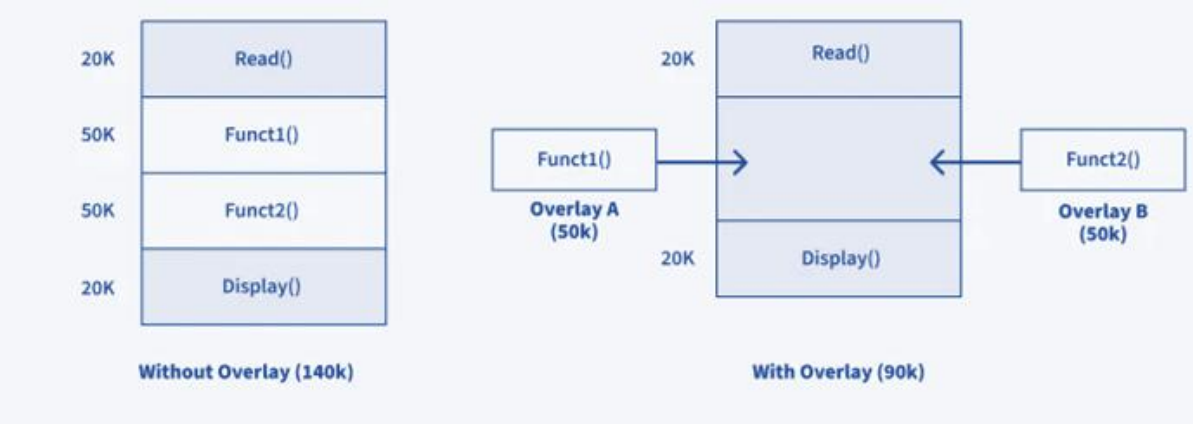
Overlay allows programs larger than memory to execute by loading only the required parts.

Concept

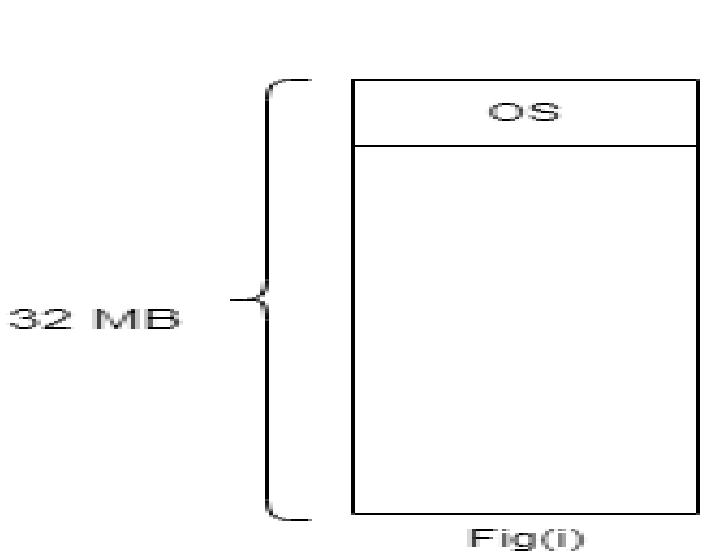
Only the required module is loaded into memory

Unused module removed

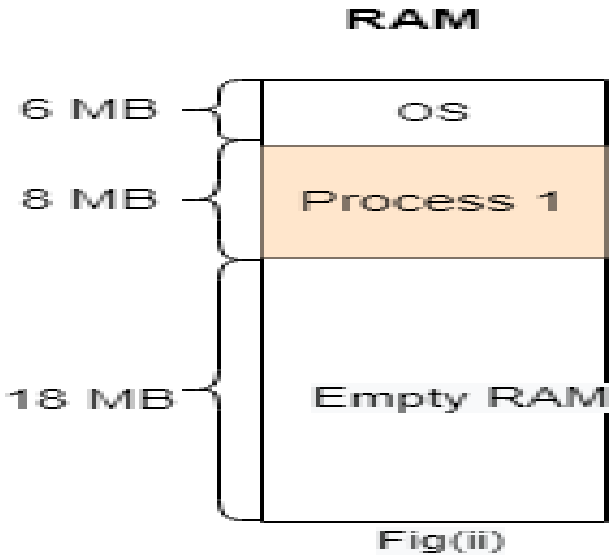
New module loaded



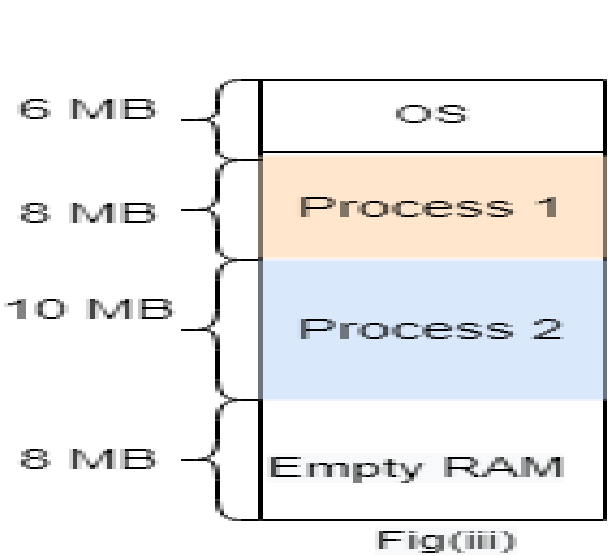
Dynamic Partitioning



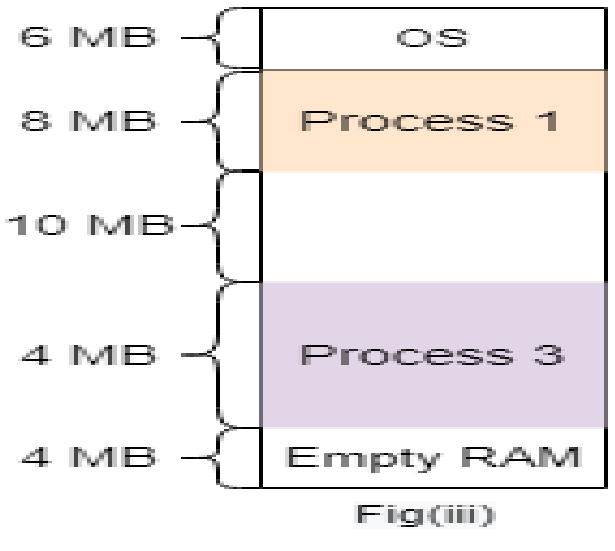
Fig(i)



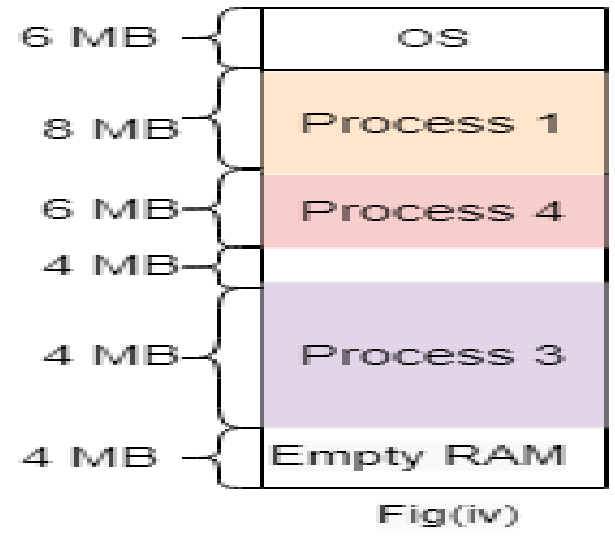
Fig(ii)



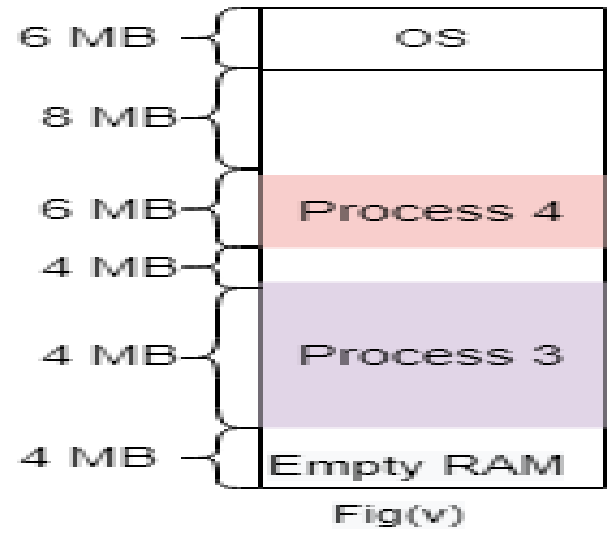
Fig(iii)



Fig(iii)



Fig(iv)



Fig(v)

Dynamic Partitioning

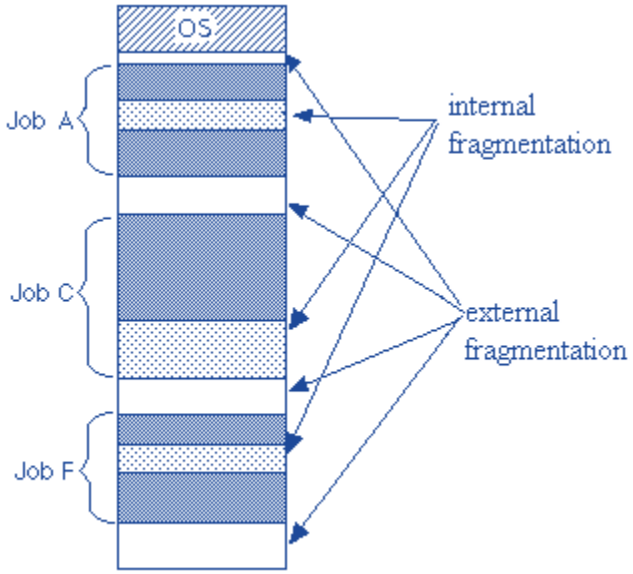
Partitions are created dynamically according to process size.

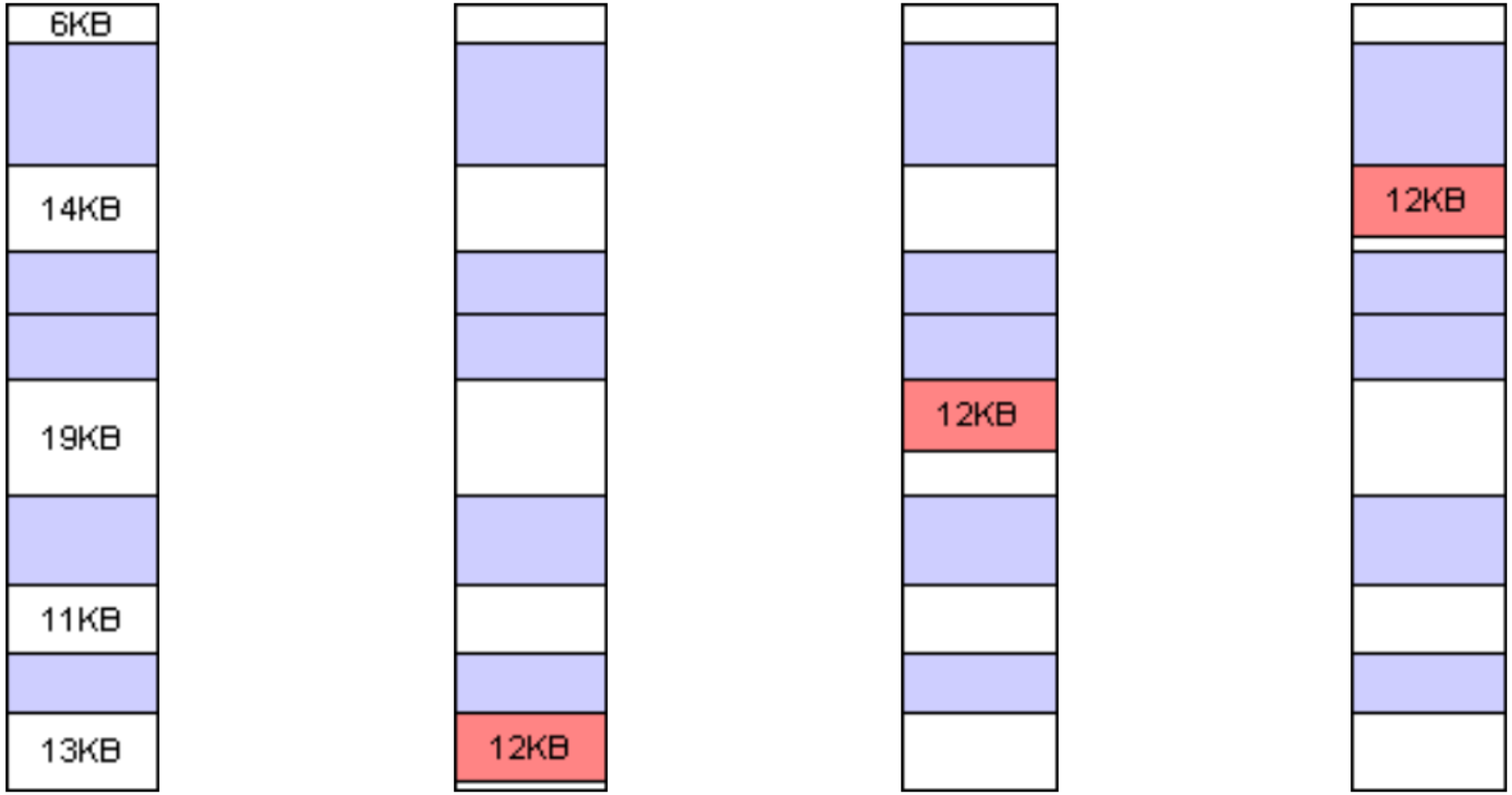
- Memory allocated when needed
- Partition size changes dynamically
- Better memory utilization

Advantages	**Disadvantages**
No internal fragmentation	External fragmentation occurs
Efficient memory utilization	Compaction may be required
Flexible allocation	Allocation complexity increases
Better support for varying process sizes	Slower allocation process

Placement Algorithms

1. First Fit
2. Best Fit
3. Worst Fit





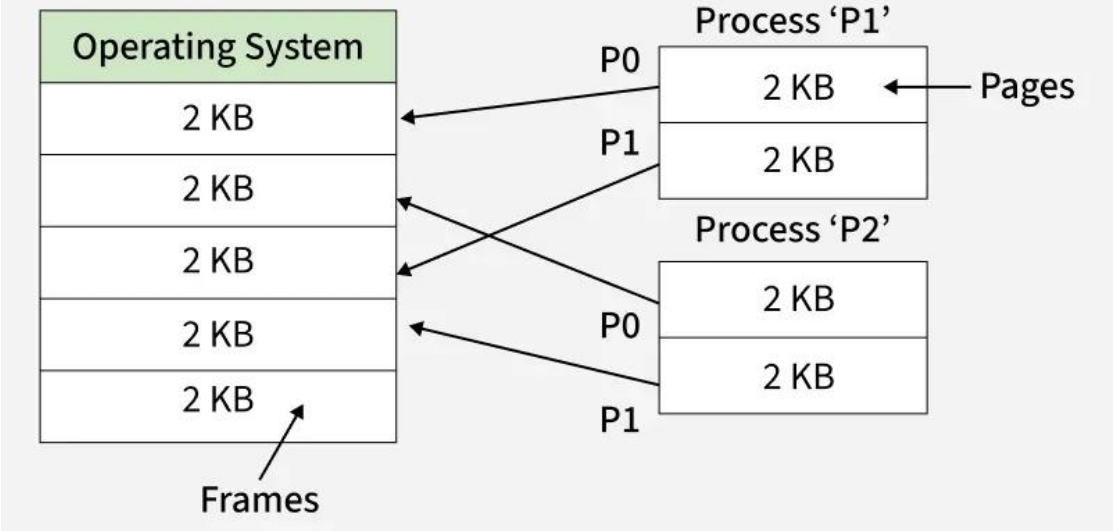
Non- Contiguous

Fragmentation Issue

Contiguous allocation leads to **external fragmentation**, where total free memory exists but isn't available in one contiguous block.

The Solution

Allowing a process's physical address space to be non-contiguous. We split the process into smaller pieces and map them to available holes.

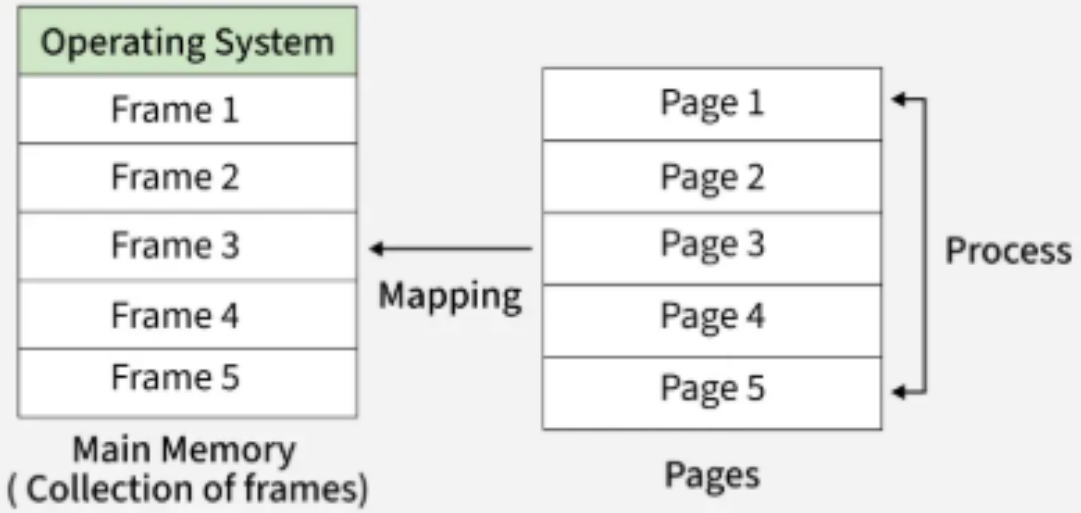


Paging

Pages: Logical memory is broken into fixed-size blocks of the same size.

Frames: Physical memory is divided into fixed-size blocks called frames.

The OS maintains a **Page Table** to translate logical pages to physical frames.



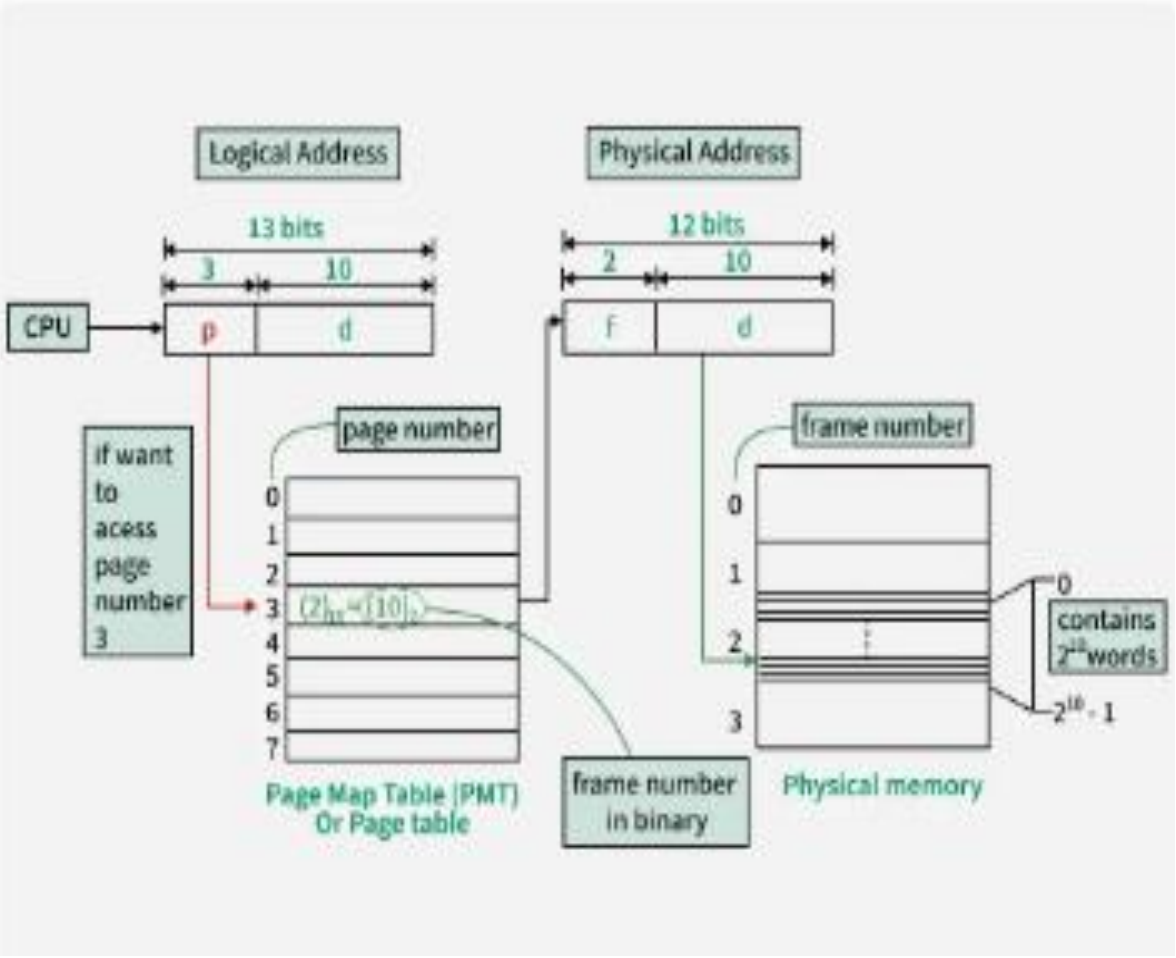
Address Translation Scheme

CPU generates a logical address divided into two parts:

- **Page Number (p):** Index into the page table.
- **Page Offset (d):** Specific location within the page/frame.

$$\text{Physical Address} = (f \times S) + d$$

Where f = Frame #, S = Page Size, d = Offset



Why Use Paging?

1. Zero External Fragmentation: Any free frame can be used for any page, regardless of its location in memory.
2. Easy Allocation: OS just needs to keep a list of free frames; no complex searching like Best-Fit or Worst-Fit.
3. Internal Fragmentation: Only occurs in the last page of a process if the process size isn't a multiple of page size.
4. Separation of Views: User sees memory as contiguous; hardware sees it as scattered frames.

Translation Lookaside Buffer (TLB)

The Problem

Standard paging requires 2 memory accesses (one for table, one for data), slowing down the CPU significantly.

The Solution (TLB)

Translation Lookaside Buffer: A fast, associative cache that stores recently used page-to-frame mappings.

Hit Ratio

If the page number is in TLB (Hit), translation is near-instant. Most programs exhibit high locality of reference.

Question

Consider a computer system with the following memory specifications:

- Logical Address Space: 32-bit virtual addresses
- Physical Memory (RAM): 16 MB (Megabytes)
- Page Size: 8 KB (Kilobytes)
- Process 'X' Size: 50 KB

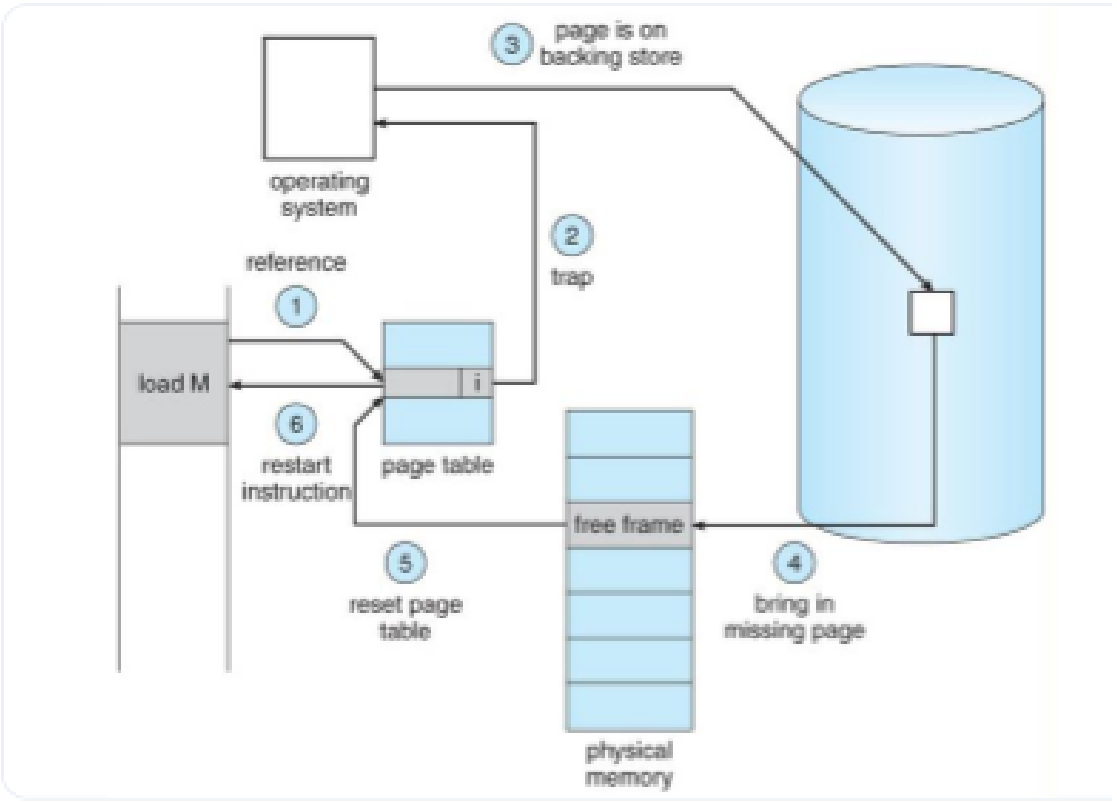
Calculate the following:

1. How many bits are required for the Page Offset (d)?
2. How many bits are required for the Page Number (p)?
3. What is the total number of Virtual Pages possible in this system?
4. How many bits are required to represent a Physical Address?
5. What is the total number of Physical Frames available in the RAM?
6. How many pages will Process 'X' require, and how much internal fragmentation (wasted space) will it experience?

What is a Page Fault?

A **Page Fault** occurs when the CPU attempts to access a page marked as 'invalid' in the page table (not in RAM).

- **Hardware Trap:** The MMU sends an interrupt to the OS.
- **Storage Fetch:** The missing page must be loaded from secondary storage (Disk/SSD).
- **Process Delay:** The process enters a 'waiting' state during I/O.



Page Replacement Algorithms :First-In, First-Out (FIFO)

- The simplest page replacement algorithm. It treats memory frames like a queue. The page that arrived first is the first one to be evicted when memory is full.

How it works: It keeps track of the order pages that are loaded. The oldest page gets the boot.

Pros: Very easy to understand and implement using a simple FIFO queue.

Cons: It doesn't care how frequently or recently a page is being used. An old page that is constantly needed will still get repeatedly evicted and reloaded.

The Catch: It suffers from Belady’s Anomaly—a strange phenomenon where giving the system more physical memory frames can actually result in more page faults.

Page Replacement Algorithms (FIFO)

The simplest algorithm: replace the page that has been in memory the longest.

Ref String	7	0	1	2	0	3	Faults
Frame 1	7	7	7	2	2	2	-
Frame 2	-	0	0	0	0	3	-
Frame 3	-	-	1	1	1	1	-
Status	Fault	Fault	Fault	Fault	Hit	Fault	**5**

Page Replacement Algorithms: Least Recently Used (LRU)

- LRU looks backward in time. It assumes that pages heavily used in the recent past will likely be used again in the near future.

How it works: It tracks when each page was last accessed. When a page fault occurs, and memory is full, it evicts the page that has sat untouched for the longest period.

Pros: Highly efficient and avoids Belady's Anomaly. It heavily mirrors real-world program behaviour (locality of reference).

Cons: Requires significant hardware support (like a clock/counter or a stack) to constantly update access times, which can slow down processing.

Page Replacement Algorithms (LRU)

0	1	7	2	3	2	7	1	0	3
0	0	0	0	3	3	3	3	0	0
	1	1	1	1	1	1	1	1	1
		7	7	7	7	7	7	7	7
			2	2	2	2	2	2	3
F	F	F	F	F	H	H	H	F	F

Page Replacement Algorithms: Optimal Page Replacement (OPR)

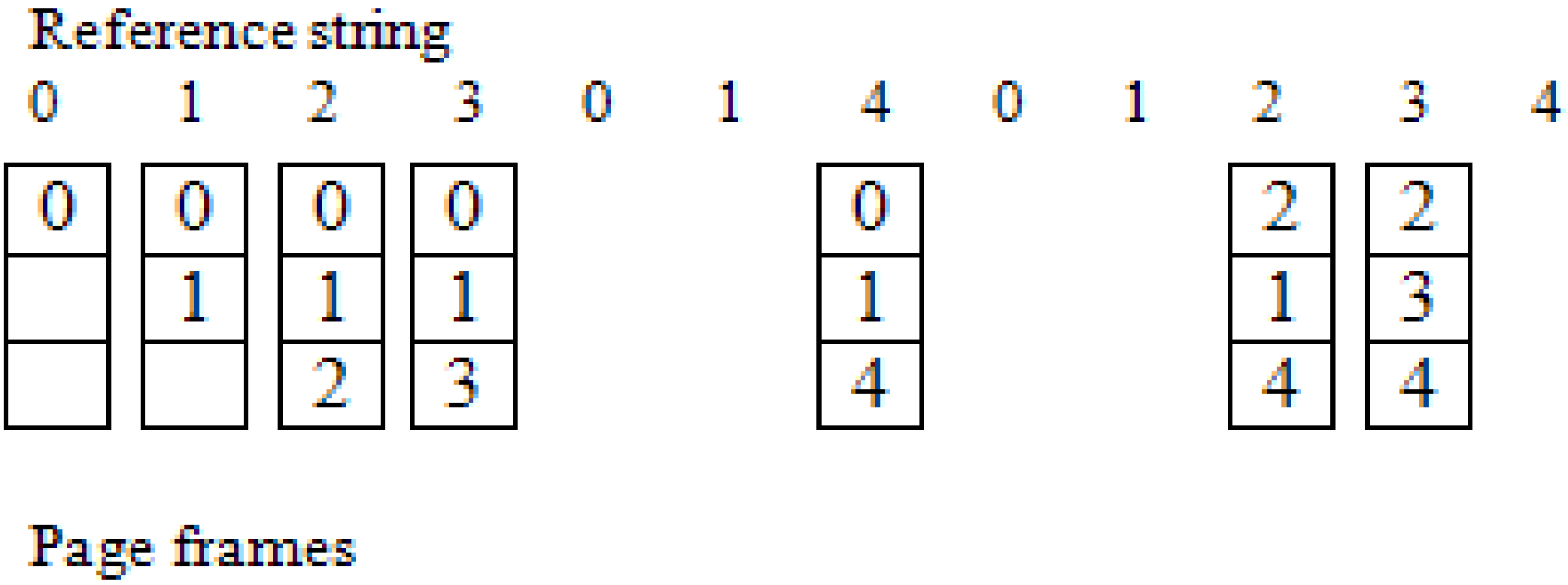
- The holy grail of page replacement. Instead of looking backward, it looks forward.

How it works: It evicts the page that will not be used for the longest period.

Pros: Guarantees the lowest possible number of page faults for any given reference string. Completely immune to Belady's Anomaly.

Cons: It is purely theoretical. An operating system cannot accurately predict the future sequence of memory requests. It is primarily used as a benchmark to measure how well other algorithms perform.

The Optimal Algorithm (OPT)



Page Replacement Algorithms: Optimal Page Replacement (OPR)

- The holy grail of page replacement. Instead of looking backward, it looks forward.

How it works: It evicts the page that will not be used for the longest period.

Pros: Guarantees the lowest possible number of page faults for any given reference string. Completely immune to Belady's Anomaly.

Cons: It is purely theoretical. An operating system cannot accurately predict the future sequence of memory requests. It is primarily used as a benchmark to measure how well other algorithms perform.

Effective Access Time (EAT)

EAT measures the actual speed of memory access including the overhead of page faults.

$$\text{EAT} = (1 - p) \times \text{ma} + p \times \text{PFST}$$

p = fault rate, ma = memory access time, PFST = fault service time.

Example: If ma = 100ns and PFST = 8ms, even a 0.1% fault rate (p=0.001) increases access time to **8,100ns** (81x slower)!

Thank you

Any Questions?